UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

Base de Datos II

Profesor Heider Sanchez

2026 - 1

Laboratorio 11: Búsqueda Eficiente de Rostros con pgvector

Introducción

Los rostros son datos no estructurados de alta dimensión: para compararlos hay que convertirlos en vectores (embeddings) y buscar los más cercanos. En este laboratorio:

1. **Vectorizarás** rostros con `face_recognition` (embeddings de 128 dimensiones) y los guardarás en PostgreSQL con la extensión **pgvector**.
2. **Buscarás** los k vecinos más cercanos de forma lineal (exacta) y con los índices **IVFFlat** y **HNSW** (aproximados).
3. **Evaluarás** los índices con **recall@k**, tomando la búsqueda lineal como referencia, y medirás cuánto tiempo ganan.
4. **Buscarás tu doble famoso:** usarás una foto tuya como consulta para encontrar a la persona del dataset que más se te parece.

> **Datos biométricos:** usa únicamente tu propia foto. No uses fotos de otras personas sin su consentimiento y borra tu embedding de la base al terminar el laboratorio.

P0. Preparación

Instalación de dependencias

Python 3.12 y las librerías:

Linux:

```
pip3 install dlib face_recognition psycopg2-binary pgvector pandas matplotlib
```

Windows:

```
conda create -n vision python=3.12
conda activate vision
conda install pandas matplotlib jupyter
conda install -c conda-forge dlib
pip install face_recognition psycopg2-binary pgvector
```

PostgreSQL con pgvector

```
docker run -d --name pgvector -e POSTGRES_PASSWORD=123456 -p 5433:5432
pgvector/pgvector:pg16
```

Dataset

Descarga LFW Dataset y descomprímelo localmente. Tiene unas 13 000 fotos, pero está **desbalanceado**: algunas personas tienen más de 500 fotos y la mayoría solo una.

```
import os, glob, time, statistics
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import face_recognition
import psycopg2
from psycopg2.extras import execute_values
from pgvector.psycopg2 import register_vector

DATASET_PATH = r"D:\temp\lfw"   # ajusta a tu ruta

coleccion = pd.DataFrame([
    {"person": os.path.basename(os.path.dirname(p)), "path": p}
    for p in glob.iglob(os.path.join(DATASET_PATH, "**", "*.jpg"), recursive=True)
])
print(len(coleccion), "imágenes")

def mostrarFotos(coleccion, posiciones):
    plt.figure(figsize=(16, 10))
    for i, idx in enumerate(posiciones):
        img = plt.imread(coleccion.path.iloc[idx])
        plt.subplot(4, 4, i + 1)
        plt.imshow(img)
        plt.title(coleccion.person.iloc[idx] + str(img.shape))
        plt.xticks([]); plt.yticks([])
    plt.tight_layout()
    plt.show()

mostrarFotos(coleccion, list(range(16)))
```

P1 (4 pts). Generar y almacenar los vectores

1. Extracción. Implementa `generate_face_embeddings(coleccion, n)` que procese las primeras `n` imágenes y devuelva, **por cada imagen en la que se detecte un rostro**, su persona, su ruta y su embedding de 128 dimensiones (las imágenes sin rostro se descartan; no pierdas la correspondencia entre embedding y foto).

```
image = face_recognition.load_image_file(filename)
encodings = face_recognition.face_encodings(image)     # lista: un vector por rostro
detectado
if encodings:
    embedding = encodings[0]                           # tomamos el primer rostro
```

> Procesar miles de imágenes tarda: guarda los resultados en la base (o en un archivo) para no repetirlo.

2. Consultas reservadas. Separa **100 imágenes** (de personas con al menos 2 fotos) que **no insertarás** en la tabla. Serán las consultas de la evaluación (P4).

3. Almacenamiento en PostgreSQL:

```
conn = psycopg2.connect(host="localhost", port=5433, user="postgres", password="123456",
dbname="postgres")
conn.autocommit = True
cur = conn.cursor()
cur.execute("CREATE EXTENSION IF NOT EXISTS vector")
register_vector(conn)               # permite enviar y recibir arreglos numpy como vector

cur.execute("""
    CREATE TABLE IF NOT EXISTS face_embeddings (
        id SERIAL PRIMARY KEY,
        name TEXT,
        path TEXT,
        embedding VECTOR(128)
    )""")

# registros = [(persona, ruta, embedding), ...]
execute_values(cur, "INSERT INTO face_embeddings (name, path, embedding) VALUES %s",
registros)
```

P2 (3 pts). Búsqueda kNN lineal y tu doble famoso

Funciones de apoyo (las usarás en todo el laboratorio):

```
def knn(q, k=10, op="<->"):
    cur.execute(f"SELECT id FROM face_embeddings ORDER BY embedding {op} %s LIMIT %s", (q,
k))
    return [r[0] for r in cur.fetchall()]

def knn_lineal(q, k=10, op="<->"):
    cur.execute("SET enable_indexscan = off")     # fuerza Seq Scan: búsqueda lineal,
aunque existan índices
    ids = knn(q, k, op)
    cur.execute("RESET enable_indexscan")
    return ids
```

a) kNN lineal. Para una imagen de consulta que no esté en la colección:

- Busca los k vecinos con distancia euclidiana `<->` y con coseno `<=>`.
- Apunta los tiempos (mediana de 10 ejecuciones).
- ¿Ambas distancias devuelven los mismos resultados? ¿Por qué?

b) Tu doble famoso. Toma **una foto tuya** (de frente y con buena luz), extrae su embedding y encuentra a las **5 personas distintas** del dataset más parecidas a ti. Como LFW tiene personas con cientos de fotos, un kNN simple devolvería varias fotos de la misma persona; por eso se agrupa por nombre:

```
def famoso_parecido(q, candidatos=50, top=5):
    cur.execute("""
        SELECT name, MIN(dist) AS dist, COUNT(*) AS fotos
        FROM (SELECT name, embedding <-> %s AS dist
              FROM face_embeddings ORDER BY embedding <-> %s LIMIT %s) t
        GROUP BY name ORDER BY dist LIMIT %s""", (q, q, candidatos, top))
    return cur.fetchall()

mi_foto = face_recognition.face_encodings(face_recognition.load_image_file("mi_foto.jpg"))
[0]
famoso_parecido(mi_foto)
```

- Muestra tu foto junto a las fotos de tus 5 dobles (usa `mostrarFotos`).
- Interpreta las distancias: `face_recognition` considera "la misma persona" por debajo de **0,6**. ¿Qué distancia obtuviste con tu doble famoso? ¿Es una coincidencia de identidad o solo un parecido?

P3 (7 pts). Búsqueda kNN con índices IVFFlat y HNSW

pgvector ofrece dos índices aproximados. Créalos **con la tabla ya cargada** y usa en la consulta **el mismo operador** que en el índice: `vector_l2_ops` ↔ `<->` y `vector_cosine_ops` ↔ `<=>`. Si no coinciden, PostgreSQL ignora el índice y hace búsqueda lineal.

```
-- IVFFlat (distancia euclidiana)
CREATE INDEX face_ivf ON face_embeddings
    USING ivfflat (embedding vector_l2_ops) WITH (lists = 100);

-- HNSW (distancia euclidiana)
CREATE INDEX face_hnsw ON face_embeddings
    USING hnsw (embedding vector_l2_ops) WITH (m = 16, ef_construction = 64);

ANALYZE face_embeddings;
```

> `lists` es el número de clusters de IVFFlat. Para estos tamaños, √N es un buen punto de partida (≈ 100 con 12 000 filas); la documentación sugiere filas/1000 para tablas de ~1 M.

Parámetros de consulta (controlan el compromiso entre velocidad y calidad):

```
SET ivfflat.probes = 10;      -- clusters visitados (IVFFlat)
SET hnsw.ef_search = 100;     -- candidatos explorados (HNSW; debe ser ≥ k)
```

Se pide:

1. Crear ambos índices y ejecutar kNN (con `knn()` y los `SET` anteriores).

2. Probar **un índice a la vez** (elimina el otro con `DROP INDEX`) para estar seguro de cuál se usa.
3. Con `EXPLAIN ANALYZE` mostrar que el plan usa `Index Scan using ...` y no `Seq Scan`, y apuntar los tiempos.
4. Repetir la búsqueda de **tu doble famoso** con cada índice: ¿obtienes los mismos 5 famosos que con la búsqueda lineal?

P4 (6 pts). Evaluación de resultados

1. Efectividad: recall@k (3 pts)

Un índice aproximado puede omitir algún vecino verdadero. Se mide con **recall@k**: la fracción de los k vecinos **exactos** (búsqueda lineal) que el índice también devuelve.

$$\text{recall@k} = \frac{|\,\text{ANN}_k \cap \text{Exacto}_k\,|}{k}$$

```
def recall_at_k(exactos, aproximados):
    return len(set(exactos) & set(aproximados)) / len(exactos)
```

Protocolo (k = 10, distancia euclidiana):

1. Para cada una de las **100 consultas reservadas**, calcula el resultado exacto con `knn_lineal()`.
2. Para cada configuración de índice, ejecuta `knn()` con `SET enable_seqscan = off` y calcula el **recall@10 promedio** y la **latencia mediana**.
3. Barre los parámetros y completa la tabla:

Índice	Parámetro	recall@10	Latencia (ms)
IVFFlat	`probes` = 1, 5, 10, 20, `lists`		
HNSW	`ef_search` = 10, 40, 100, 200		

Se pide:

- Graficar **recall@10 vs. latencia** para ambos índices. ¿Cuál se acerca más a la búsqueda lineal con menor tiempo?
- Explicar qué ocurre con `probes = lists` y por qué.
- Interpretar por qué un índice aproximado puede diferir de la búsqueda lineal (IVF: vecinos en otro cluster; HNSW: búsqueda voraz sobre un grafo).

2. Rendimiento (2 pts)

- Probar con **1000, 2000, 4000, 8000 y 12000** embeddings (cargando los primeros N y recreando los índices).
- Para cada tamaño y cada índice, usar los parámetros **más baratos que logren recall@10 ≥ 0,95**, de modo que la comparación sea con calidad equivalente.
- Medir con `EXPLAIN ANALYZE` (mediana de 10 ejecuciones, descartando la primera).
- En **un solo gráfico**, comparar el tiempo de consulta de **kNN lineal, kNN con IVFFlat y kNN con HNSW** según el tamaño del dataset.

3. Conclusiones (1 pt)

- Ventajas y desventajas de cada método (tiempo de construcción, memoria, recall, parámetros).
- ¿Cómo escalan estas técnicas con datasets de millones de vectores?
- Dos casos reales distintos a reconocimiento facial que se beneficien de una base de datos vectorial.

Entregable

Informe (con el notebook) que incluya, por apartado: el código, las tablas, los gráficos y las capturas de `EXPLAIN ANALYZE`. En P2 y P3 incluye el resultado de tu doble famoso (no es obligatorio mostrar tu foto en el informe).